Lab 4: Vibe Coding — Maze Runner

LLM Interaction History & Code Review Transcript

Conversation ID: 4ff3743a-7a5e-4c7e-be57-12906d399426
Model: Gemini 3.8 Flash (Vibe Coding Assistant)
Deliverable: Lab-4 Chat History Export (PDF & DOCX)
Repository: 12_maze_runner-main (Pygame)

Turn 1 — User Prompt

okay ive run the game as mentioned in the student doc and ive found that player can just pass through walls theres no grid wall checking only outer boundary. fix the player and game file accordingly

Assistant Response

I have updated the wall collision detection in the player module so that the player cannot pass through maze grid walls.

Summary of Changes

1. **player.py**:

• Enhanced `_hits_wall()` method to perform grid-level wall checking against the maze data `walls[r][c]`.

• Checks North (`w[0]`), South (`w[1]`), East (`w[2]`), and West (`w[3]`) wall states for all maze grid cells overlapping the player's bounding box (`rect`).

• Retained and verified outer boundary checking against maze bounds (`cols * CELL` and `rows * CELL`).

2. **Verification**:

• Ran automated test suites checking all 4 directional movement keys (W/A/S/D and arrow keys) against locked and procedurally generated maze layouts. Verified that the player is blocked by solid walls while passing through open corridors without sticking.

Turn 2 — User Prompt

Add a BFS solver to my pygame maze. When H is pressed, highlight the shortest path from start to exit by drawing colored squares on each cell in the path. Toggle it off on second press.

Assistant Response

I have implemented the BFS shortest path solver and path highlighting feature with toggle functionality.

Summary of Changes

1. **game/maze.py**:

• Added `solve_maze(walls, start, end)` using Breadth-First Search (BFS) with `collections.deque`.

• Traverses available passages (where wall flags are `False`) to compute the exact shortest sequence of `(row, col)` cells from the start cell `(0, 0)` to the exit cell `(ROWS-1, COLS-1)`.

2. **game/game_engine.py**:

• Added `self.show_path` boolean toggle state and computed `self.solution_path` upon maze initialization/reset.

• Handled the `H` key press event (`pygame.K_h`) in `handle_events()` to toggle `self.show_path`.

• Added `draw_path()` to render gold/yellow highlighted squares on all cells belonging to the solution path when active.

• Updated HUD text display to include `H = Hint`.

Turn 3 — User Prompt

Add fog of war to my maze game. Draw a dark overlay over the whole maze surface, then cut out a circular transparent region around the player using pygame.draw.circle on a surface with SRCALPHA. Reveal only nearby cells.

Assistant Response

I have added the Fog of War feature to the maze game.

Summary of Changes

1. **game/game_engine.py**:

• Added `draw_fog()` which creates a dark `pygame.Surface` initialized with SRCALPHA and filled with dark overlay color `(10, 10, 20, 255)`.

• Uses `pygame.draw.circle(fog_surf, (0, 0, 0, 0), self.player.rect.center, 3 * CELL)` to carve out a circular transparent region revealing cells within a 3-cell radius around the player.

• Blits `fog_surf` over the maze region in `draw()`.

• Added keyboard toggle functionality via the F key (`pygame.K_f`), defaulting to active upon game reset and updating the HUD label to `F=Fog`.

Turn 4 — User Prompt

After the player solves the maze, save their time to a list (up to 5 entries, sorted ascending). Display the leaderboard on the win screen. Persist it in a JSON file called leaderboard.json.

Assistant Response

I have added the timer leaderboard persisted in `leaderboard.json` and displayed on the win screen.

Summary of Changes

1. **game/game_engine.py**:

• Added `load_leaderboard()` and `save_leaderboard()` to read and write completion times to `leaderboard.json`.

• On reaching the exit in `update()`, the final elapsed time is recorded, appended to the leaderboard, sorted in ascending order, trimmed to the top 5 entries, and saved.

• Updated the win screen overlay in `draw()` to display the top 5 leaderboard rankings, highlighting the player's current completion time.

Turn 5 — User Prompt

Add a difficulty select screen before the maze generates. Show three buttons (Easy, Medium, Hard) with different COLS/ROWS values. Let the player click to select, then launch the maze with those dimensions.

Assistant Response

I have added the difficulty selection screen prior to maze generation.

Summary of Changes

1. **game/game_engine.py**:

• Added game state management starting in `SELECT` mode before generating the maze.

• Built `draw_select()` with buttons and hover effects for three difficulty tiers:

• **Easy**: 10 × 8 (`cols=10, rows=8`)

• **Medium**: 15 × 13 (`cols=15, rows=13`)

• **Hard**: 20 × 18 (`cols=20, rows=18`)

• Implemented `start_game(cols, rows)` to dynamically adapt the window size (`cols * CELL` by `rows * CELL + 60`) and generate the maze with the selected dimensions.

• Added `M` / `ESC` shortcut to return to the difficulty select screen at any time.

2. **README.md**:

• Updated the Controls table and marked completed tasks in the submission checklist.

Turn 6 — User Prompt

append the chat link to the readme

Assistant Response

I have appended the chat session link and conversation ID to README.md under both the **Submission Checklist** and the new **LLM Chat History** section.

Turn 7 — User Prompt

also export the chat as a pdf or a doc

Assistant Response

Exporting the complete chat history as both Chat_History_Lab4.pdf and Chat_History_Lab4.docx.